Mathematical Reference

Basis Function Expansion for Gravitational Potentials

`bfeax` — JAX implementation

Contents

1 Overview **3**

2 Radial Grid **3**
- 2.1 Log-uniform spacing . . . 3
- 2.2 Parameter guidance . . . 3

3 Real Spherical Harmonics **4**
- 3.1 Convention . . . 4
- 3.2 Associated Legendre recurrence . . . 4

4 Angular Projection **4**
- 4.1 Spherical harmonic decomposition at each shell . . . 4
- 4.2 Gauss–Legendre $\times$ uniform-φ quadrature . . . 5
- 4.3 Default quadrature orders . . . 5

5 Radial Poisson Solver **5**
- 5.1 Green's function solution . . . 5
- 5.2 Trapezoidal discretisation . . . 6

6 Boundary Corrections **6**
- 6.1 Inner tail correction ($0 \to r_{\min}$) . . . 6
- 6.2 Outer tail correction ($r_{\max} \to \infty$) . . . 6

7 Inner Cusp Subtraction **7**
- 7.1 Motivation . . . 7
- 7.2 Procedure . . . 7
- 7.3 Guard . . . 7

8 Spline Interpolation **7**
- 8.1 Natural cubic splines in $\log r$ . . . 7
- 8.2 Fitting . . . 8
- 8.3 Batched fitting . . . 8

8.4 Evaluation 8

9 Potential Evaluation **8**
9.1 Coordinate conversion 8
9.2 Stacked evaluation path 8
9.3 Accelerations 9

10 Scan-based Poisson Solve **9**
10.1 Motivation 9
10.2 Implementation 9

11 Convergence **9**

12 Module Summary **10**

1 Overview

The code converts an arbitrary density function $\rho(\mathbf{x})$ into a smooth gravitational potential $\Phi(\mathbf{x})$ by decomposing in spherical harmonics angularly and using the radial Green's function for Poisson's equation. The pipeline has four stages:

1. Build a log-spaced radial grid $\{r_i\}$.
2. At each shell r_i, project ρ onto spherical harmonics to obtain the radial coefficients $\rho_{\ell m}(r_i)$.
3. Solve the radial Poisson equation for each (ℓ, m) mode to obtain $\Phi_{\ell m}(r_i)$ using a Green's function with analytic boundary corrections.
4. Fit natural cubic splines to $\Phi_{\ell m}(r)$ and $\rho_{\ell m}(r)$ for continuous evaluation.

The potential and reconstructed density are then evaluated as

$$\Phi(\mathbf{x}) = \sum_{\ell=0}^{\ell_{\max}} \sum_{m=-\ell}^{\ell} \Phi_{\ell m}(r)\, Y_{\ell m}(\theta, \varphi), \tag{1}$$

$$\rho_{\mathrm{rec}}(\mathbf{x}) = \sum_{\ell=0}^{\ell_{\max}} \sum_{m=-\ell}^{\ell} \rho_{\ell m}(r)\, Y_{\ell m}(\theta, \varphi), \tag{2}$$

where (r, θ, φ) are standard spherical coordinates. The gravitational acceleration is $\mathbf{a} = -\nabla\Phi$, computed via automatic differentiation.

2 Radial Grid

2.1 Log-uniform spacing

All radial integrals are discretised on a grid of N_r points uniformly spaced in $\log r$:

$$r_i = \exp\left(\log r_{\min} + \frac{i}{N_r - 1}\big(\log r_{\max} - \log r_{\min}\big)\right), \quad i = 0, \ldots, N_r - 1. \tag{3}$$

Log spacing allocates equal resolution per decade, which is appropriate for power-law profiles (e.g. NFW, Hernquist) that vary by many orders of magnitude across the radial range.

2.2 Parameter guidance

Parameter	Typical value	Notes
$r_{\min}$	$\sim 10^{-2}\, r_s$	Well inside the inner cusp
$r_{\max}$	$\geq 30\, r_s$	Critical for NFW; outer integral diverges logarithmically
N_r	64–128	128 gives $\lesssim 0.1\%$ for NFW

3 Real Spherical Harmonics

3.1 Convention

The code uses *real orthonormal* spherical harmonics. Starting from the complex harmonics Y_ℓ^m, the real combinations are

$$Y_{\ell m}(\theta,\varphi) = \begin{cases} \sqrt{2}\, N_{\ell m}\, P_\ell^{|m|}(\cos\theta)\, \cos(m\varphi) & m > 0, \\ N_{\ell 0}\, P_\ell^0(\cos\theta) & m = 0, \\ \sqrt{2}\, N_{\ell |m|}\, P_\ell^{|m|}(\cos\theta)\, \sin(|m|\varphi) & m < 0, \end{cases} \tag{4}$$

with normalisation

$$N_{\ell m} = \sqrt{\frac{2\ell+1}{4\pi}\frac{(\ell-|m|)!}{(\ell+|m|)!}}. \tag{5}$$

These satisfy the orthonormality relation

$$\int Y_{\ell m}\, Y_{\ell' m'}\, \mathrm{d}\Omega = \delta_{\ell\ell'}\, \delta_{mm'}. \tag{6}$$

3.2 Associated Legendre recurrence

The associated Legendre polynomials $P_\ell^m(x)$ are computed via the three-term recurrence (evaluated at $x = \cos\theta$). The seed and diagonal step are

$$P_0^0 = 1, \tag{7}$$

$$P_\ell^\ell = -(2\ell-1)\sin\theta\, P_{\ell-1}^{\ell-1}, \tag{8}$$

$$P_\ell^{\ell-1} = (2\ell-1)\cos\theta\, P_{\ell-1}^{\ell-1}, \tag{9}$$

and the upward recurrence for $m \le \ell - 2$ is

$$P_\ell^m = \frac{(2\ell-1)\cos\theta\, P_{\ell-1}^m - (\ell+m-1)\, P_{\ell-2}^m}{\ell - m}. \tag{10}$$

All (ℓ, m) pairs with $0 \le m \le \ell \le \ell_{\max}$ are computed in a single sweep; the loop structure is unrolled at JAX trace time since $\ell_{\max}$ is a static Python integer.

4 Angular Projection

4.1 Spherical harmonic decomposition at each shell

For each radius r_i, the density is integrated over the unit sphere:

$$\rho_{\ell m}(r_i) = \int \rho(r_i,\theta,\varphi)\, Y_{\ell m}(\theta,\varphi)\, \mathrm{d}\Omega. \tag{11}$$

This is the fundamental step that converts the 3D density into a set of 1D radial functions, one per mode (ℓ, m).

4.2 Gauss–Legendre × uniform-φ quadrature

The integral over the sphere is evaluated numerically as a product quadrature. In the $\cos\theta$ direction, N_θ Gauss–Legendre nodes $\{x_j, w_j\}$ are used (where $x_j = \cos\theta_j$ and w_j are the GL weights in $\mathrm{d}(\cos\theta)$). In the φ direction, N_φ equally spaced nodes with weight $\Delta\varphi = 2\pi/N_\varphi$ are used. The discrete approximation is

$$\rho_{\ell m}(r_i) \approx \sum_{j=1}^{N_\theta} \sum_{k=1}^{N_\varphi} \rho(r_i, \theta_j, \varphi_k)\, Y_{\ell m}(\theta_j, \varphi_k)\, w_j\, \Delta\varphi. \tag{12}$$

Note that the GL rule integrates in $\mathrm{d}(\cos\theta)$, so the $\sin\theta$ Jacobian factor is *not* included explicitly; it is absorbed into the GL weights.

The angular quadrature is pre-evaluated once as a tensor $Y_{\ell m}^{jk} \equiv Y_{\ell m}(\theta_j, \varphi_k)$ of shape $(N_{\mathrm{modes}}, N_\theta, N_\varphi)$, and the projection over all radii is computed in a single `jnp.einsum`:

$$\rho_{\ell m}(r_i) \approx \sum_{j,k} \rho(r_i, \theta_j, \varphi_k)\, Y_{\ell m}^{jk}\, w_j\, \Delta\varphi. \tag{13}$$

4.3 Default quadrature orders

The Gauss–Legendre rule with N_θ points integrates polynomials of degree $2N_\theta - 1$ exactly. For non-polynomial integrands (e.g. NFW), a factor of 3 margin is used:

$$N_\theta = 3(\ell_{\max} + 2), \qquad N_\varphi = 4\ell_{\max} + 7 \quad \text{(kept odd to avoid aliasing).} \tag{14}$$

5 Radial Poisson Solver

5.1 Green's function solution

For each (ℓ, m) mode, the radial Poisson equation

$$\frac{1}{r^2}\frac{\mathrm{d}}{\mathrm{d}r}\left(r^2 \frac{\mathrm{d}\Phi_{\ell m}}{\mathrm{d}r}\right) - \frac{\ell(\ell+1)}{r^2}\Phi_{\ell m} = 4\pi G\, \rho_{\ell m} \tag{15}$$

is solved via the standard Green's function (with $G = 1$):

$$\boxed{\Phi_{\ell m}(r) = -\frac{4\pi}{2\ell+1}\left[r^{-(\ell+1)} I_{\mathrm{in}}(r) + r^{\ell} I_{\mathrm{out}}(r)\right],} \tag{16}$$

where the two cumulative integrals are

$$I_{\mathrm{in}}(r) = \int_0^r \rho_{\ell m}(r')\, r'^{\,\ell+2}\, \mathrm{d}r', \tag{17}$$

$$I_{\mathrm{out}}(r) = \int_r^\infty \rho_{\ell m}(r')\, r'^{\,1-\ell}\, \mathrm{d}r'. \tag{18}$$

I_{in} is a running integral accumulated from small r outward ("inside-out"); I_{out} is accumulated from large r inward ("outside-in"). Both are evaluated by the trapezoidal rule on the log-spaced grid.

5.2 Trapezoidal discretisation

Let $f_i^{\rm in} = \rho_{\ell m}(r_i)\, r_i^{\ell+2}$ and $\Delta r_i = r_{i+1} - r_i$. Then

$$I_{\rm in}(r_i) = \Delta I_{\rm in} + \sum_{j=0}^{i-1} \tfrac{1}{2}\big(f_j^{\rm in} + f_{j+1}^{\rm in}\big)\Delta r_j, \tag{19}$$

where $\Delta I_{\rm in}$ is the analytic inner tail correction (Section 6). The outside-in sum for $I_{\rm out}$ is constructed analogously via a reversed cumulative sum.

6 Boundary Corrections

The grid covers only $[r_{\rm min}, r_{\rm max}]$, but the integrals in equations (17)–(18) extend to 0 and ∞ respectively. Both truncations can introduce significant bias, particularly for the NFW outer integral which diverges logarithmically unless $r_{\rm max}$ is very large.

6.1 Inner tail correction ($0 \to r_{\rm min}$)

The power-law slope of $\rho_{\ell m}$ near $r_{\rm min}$ is estimated from the first three grid points via a log–log finite difference:

$$\alpha_{\rm in} \approx \left\langle \frac{\Delta \log |\rho_{\ell m}|}{\Delta \log r} \right\rangle_{r_0, r_1, r_2}. \tag{20}$$

Assuming $\rho_{\ell m}(r) \approx A_{\rm in}\, r^{\alpha_{\rm in}}$ for $r < r_{\rm min}$, the inner integral contribution is

$$\Delta I_{\rm in} = \int_0^{r_{\rm min}} A_{\rm in}\, r^{\alpha_{\rm in}}\, r^{\ell+2}\, {\rm d}r = \frac{A_{\rm in}\, r_{\rm min}^{\alpha_{\rm in}+\ell+3}}{\alpha_{\rm in} + \ell + 3}, \tag{21}$$

which converges provided $\alpha_{\rm in} + \ell + 3 > 0$ (always satisfied for physical density cusps with $\alpha_{\rm in} > -3$). The correction is added as a constant to $I_{\rm in}(r)$ for all $r \geq r_{\rm min}$.

The amplitude is recovered as $A_{\rm in} = {\rm sign}(\rho_{\ell m}(r_0)) \cdot |\rho_{\ell m}(r_0)| \cdot r_0^{-\alpha_{\rm in}}$, using `exp(alpha * log(r))` to avoid overflow when $\alpha_{\rm in}$ is a traced JAX scalar.

6.2 Outer tail correction ($r_{\rm max} \to \infty$)

Similarly, the slope at the outer boundary is estimated from the last three grid points:

$$\alpha_{\rm out} \approx \left\langle \frac{\Delta \log |\rho_{\ell m}|}{\Delta \log r} \right\rangle_{r_{N_r-3}, r_{N_r-2}, r_{N_r-1}}. \tag{22}$$

Assuming $\rho_{\ell m}(r) \approx A_{\rm out}\, r^{\alpha_{\rm out}}$ for $r > r_{\rm max}$,

$$\Delta I_{\rm out} = \int_{r_{\rm max}}^{\infty} A_{\rm out}\, r^{\alpha_{\rm out}}\, r^{1-\ell}\, {\rm d}r = \frac{A_{\rm out}\, r_{\rm max}^{2-\ell}}{\ell - \alpha_{\rm out} - 2}, \tag{23}$$

which converges provided $\alpha_{\rm out} < \ell - 2$. For the monopole $\ell = 0$ this requires $\alpha_{\rm out} < -2$; for NFW ($\alpha_{\rm out} \approx -3$) this is satisfied for all $\ell \geq 0$.

Guard conditions. Both corrections are applied only when

- the boundary value $|\rho_{\ell m}(r_{\text{boundary}})|$ exceeds 10^{-8} of the per-mode maximum (inner), or $\rho_{\ell m} > 0$ (outer), and
- the power-law integral converges.

All conditionals use `jnp.where` to remain JIT-safe; neither branch propagates NaNs.

7 Inner Cusp Subtraction

7.1 Motivation

For profiles with a steep inner cusp (e.g. NFW: $\rho \sim r^{-1}$), the radial coefficients $\rho_{\ell m}(r)$ diverge toward $r = 0$. Fitting a cubic spline directly to such data is poorly conditioned near $r_{\min}$: small rounding errors in $\rho_{\ell m}(r_{\min})$ lead to large oscillations in the fitted spline.

7.2 Procedure

For each mode (ℓ, m):

1. Estimate the inner power-law parameters $(\alpha_{\ell m}, A_{\ell m})$ from the first three grid points (same finite-difference estimator as in Section 6).
2. Define the background $\text{bg}(r) = A_{\ell m}\, r^{\alpha_{\ell m}}$.
3. Fit the cubic spline to the *residual* $\widetilde{\rho}_{\ell m}(r) = \rho_{\ell m}(r) - \text{bg}(r)$, which is smooth and nearly flat near $r_{\min}$.
4. At evaluation time, reconstruct $\rho_{\ell m}(r) = \widetilde{\rho}_{\ell m}(r) + A_{\ell m}\, r^{\alpha_{\ell m}}$.

7.3 Guard

The subtraction is only applied when

$$|\rho_{\ell m}(r_{\min})| > 10^{-6} \cdot \rho_{\text{scale}}, \tag{24}$$

where $\rho_{\text{scale}} = \max_{\ell m} \max_i |\rho_{\ell m}(r_i)|$ is the *global* amplitude scale across all modes. This prevents spurious large backgrounds on symmetry-zero modes (e.g. odd-ℓ modes of an octant-symmetric density) whose values sit at machine-epsilon level.

8 Spline Interpolation

8.1 Natural cubic splines in $\log r$

Both $\Phi_{\ell m}(r)$ and $\rho_{\ell m}(r)$ are stored as natural cubic splines parameterised in $u = \log r$. On each interval $[u_i, u_{i+1}]$, the spline takes the form

$$S(u) = a_i + b_i\,(u - u_i) + c_i\,(u - u_i)^2 + d_i\,(u - u_i)^3. \tag{25}$$

8.2 Fitting

The coefficients are determined by requiring S to be C^2 everywhere, with natural boundary conditions $S''(u_0) = S''(u_{N_r-1}) = 0$. This leads to the tridiagonal system for the second derivatives $M_i = S''(u_i)$:

$$h_{i-1}\, M_{i-1} + 2(h_{i-1} + h_i)\, M_i + h_i\, M_{i+1} = 3\left(\frac{y_{i+1} - y_i}{h_i} - \frac{y_i - y_{i-1}}{h_{i-1}}\right), \tag{26}$$

where $h_i = u_{i+1} - u_i$ and y_i are the function values. This is solved via `jnp.linalg.solve` on the $(N_r - 2) \times (N_r - 2)$ dense matrix (safe to JIT since array shapes are static). The polynomial coefficients follow as

$$a_i = y_i, \tag{27}$$

$$b_i = \frac{y_{i+1} - y_i}{h_i} - \frac{h_i\,(2M_i + M_{i+1})}{3}, \tag{28}$$

$$c_i = M_i, \tag{29}$$

$$d_i = \frac{M_{i+1} - M_i}{3\,h_i}. \tag{30}$$

8.3 Batched fitting

When building the expansion via the fast (spheroid) path, all N_{modes} splines share the same $\log r$ knots. A single matrix factorisation of the $(N_r - 2) \times (N_r - 2)$ system is applied to the full right-hand side matrix of shape $(N_r - 2) \times N_{\text{modes}}$ simultaneously, avoiding redundant factorisations.

8.4 Evaluation

For a query $u^* = \log r^*$, the interval index is found via binary search (`jnp.searchsorted`), then the cubic polynomial is evaluated with Horner's method. Out-of-range queries are clamped to the nearest boundary interval.

9 Potential Evaluation

9.1 Coordinate conversion

Cartesian coordinates (x, y, z) are converted to spherical (r, θ, φ):

$$r = \|\mathbf{x}\|, \quad \theta = \arccos\left(\frac{z}{r}\right), \quad \varphi = \operatorname{atan2}(y, x) \bmod 2\pi. \tag{31}$$

9.2 Stacked evaluation path

For expansions built via the fast path (`from_spheroid` or `ExpansionGrid`), all mode splines are stored as batched coefficient arrays of shape $(N_{\text{modes}}, N_r - 1)$. A single `searchsorted` call finds the radial interval, and the polynomial is evaluated for all modes simultaneously:

$$\Phi_{\ell m}(r^*) = a_{\ell m,\,i} + b_{\ell m,\,i}\,\delta u + c_{\ell m,\,i}\,\delta u^2 + d_{\ell m,\,i}\,\delta u^3, \quad \delta u = \log r^* - u_i. \tag{32}$$

The final summation is a single dot product:

$$\Phi(\mathbf{x}) = \sum_{\ell m} \Phi_{\ell m}(r^*)\, Y_{\ell m}(\theta, \varphi) = \mathbf{\Phi}_{\text{modes}} \cdot \mathbf{Y}_{\text{modes}}. \tag{33}$$

9.3 Accelerations

Gravitational accelerations are obtained via automatic differentiation:

$$\mathbf{a}(\mathbf{x}) = -\nabla_{\mathbf{x}}\, \Phi(\mathbf{x}), \tag{34}$$

implemented as `jax.vmap(jax.grad(__call__))`. No analytical gradient formula is required.

10 Scan-based Poisson Solve

10.1 Motivation

Inside the JIT-compiled build functions, the naive Python loop `for l in range(l_max+1)` causes XLA to unroll $\ell_{\max} + 1$ separate subgraphs — one per ℓ value — each with statically known exponents. For large $\ell_{\max}$ this inflates compilation time significantly.

10.2 Implementation

The loop is replaced by `jax.lax.scan` over all $N_{\text{modes}} = (\ell_{\max} + 1)^2$ modes in a flat ordering. A precomputed array $\ell_{\text{mode}}[k]$ stores the ℓ value for the k-th mode. The scan body processes one column $\rho_{\ell m}(\cdot)$ per step:

```
l_per_mode = [l for l in range(l_max+1) for _ in range(2*l+1)]
_, phi_T = jax.lax.scan(solve_one, None, (rho_lm_all.T, l_per_mode))
```

Because ℓ is now a traced float, the power-law terms are computed as

$$r^{\ell+2} \;\longrightarrow\; \exp\big((\ell + 2)\log r\big), \tag{35}$$

which is numerically equivalent and avoids Python-level specialisation. XLA compiles a single generic body and amortises it over all steps, yielding substantially shorter compilation time for large $\ell_{\max}$.

11 Convergence

For a triaxial NFW halo ($q_1 = 0.8$, $q_2 = 0.5$) with $N_r = 128$, $r_{\max} = 300\, r_s$, the density reconstruction error decreases geometrically:

$\ell_{\max}$	Median relative error
4	$\sim 3\%$
6	$\sim 0.8\%$
8	$\sim 0.2\%$
10	$\sim 0.06\%$

The error drops approximately $4\times$ per $\Delta\ell = 2$, consistent with geometric (spectral) convergence in the angular expansion. The potential converges faster than the density; $\ell_{\max} = 2$–4 is often sufficient for potential applications.

12 Module Summary

File	Responsibility
`grid.py`	Log-spaced radial grid
`sph_harm.py`	Real $Y_{\ell m}$ via Legendre recurrence; GL $\times$ uniform quadrature grid
`density_coeffs.py`	Angular projection $\rho \to \rho_{\ell m}(r_i)$ via `vmap` + `einsum`
`spline.py`	Natural cubic splines in JAX (JIT-safe, batched fitting)
`poisson.py`	Green's function radial Poisson solver with boundary corrections
`potential.py`	`MultipoleExpansion` class; fast-path `jax.lax.scan` Poisson solve